hft-ipc-bench Benchmark Report

Table of contents

1 Introduction **2**

2 Testing Methodology **2**
- 2.1 SHM Result Preprocessing . . . 2
- 2.2 System Requirement . . . 2
- 2.3 Dependencies . . . 3
- 2.4 Build Process . . . 3
- 2.5 Test Execution . . . 4
- 2.6 Test Case Matrix . . . 4
- 2.7 Metrics . . . 4

3 Latency Percentile Summary by Consumer Count and Target MPS **5**

4 P50 Latency Scaling by Consumer Count **6**
- 4.1 Interpretation of the UDS and gRPC Consumer-Count Dip . . . 7

5 P50 Latency vs Target MPS at Fixed Consumer Counts **8**

6 Throughput by Target MPS and Consumer Count **9**

7 Throughput vs P50 Latency **13**

8 Loss by Target MPS **14**

9 Result Summary and Conclusion **15**
- 9.1 Executive Summary . . . 15
- 9.2 Key Findings . . . 15
- 9.3 Recommendations . . . 16

1 Introduction

This report evaluates several in-host inter-process communication (IPC) mechanisms for a high-frequency trading style market data fan-out workload. The benchmark models a single producer distributing fixed-size 64-byte messages to one or more local consumers. The goal is to compare typical latency, throughput capacity, and fan-out scaling behavior as the number of consumers increases.

The tested implementations are:

- **SHM (lock-free)**: a custom shared-memory ring-buffer implementation using POSIX shared memory and atomic coordination.
- **UDS stream**: a Unix domain socket baseline using one `SOCK_STREAM` connection per consumer and fixed-size 64-byte message framing.
- **ZeroMQ PUB/SUB**: a brokerless publish/subscribe baseline over local IPC transport.
- **gRPC streaming**: a server-streaming RPC implementation using Protocol Buffers and gRPC C++.

The analysis reads `output/analysis_summary.csv`. Because the benchmark was run inside a VM, tail latency can be affected by virtualization noise, vCPU scheduling, and host-side interference. For this reason, **P50 latency is treated as the primary latency metric**, while P90 and P99 are reported as supporting indicators rather than absolute HFT-grade tail-latency evidence.

2 Testing Methodology

2.1 SHM Result Preprocessing

SHM results are aggregated from multiple benchmark rounds rather than a single run. To reduce VM-induced noise, any SHM row with abnormal latency is removed before aggregation; in this report, rows with `p50_ns > 1000` are treated as outliers and the entire row is dropped. The remaining rows with the same configuration are averaged into `shm_final.csv`, and that preprocessed SHM summary is then merged into `analysis_summary.csv` for the report.

2.2 System Requirement

Item	Specification
CPU	Intel(R) Core(TM) i5-12400, 2.5 GHz
Memory	32.0 GB
OS	Ubuntu 22.04 LTS running in a VM, x86-64

Item	Specification
Compiler	g++
Permission	sudo

2.3 Dependencies

Dependency Package	Purpose / Description
`build-essential`, `g++`, `make`, `pkg-config`	C++ build toolchain
`libzmq3-dev`	ZeroMQ C++ library
`libgrpc++-dev`	gRPC C++ library
`libprotobuf-dev`	Protobuf runtime
`protobuf-compiler`	`protoc` code generator
`protobuf-compiler-grpc`	gRPC plugin for `protoc`
`python3`, `python3-pip`	Runtime for analysis scripts

Install command:

```
sudo apt-get install -y \
    libzmq3-dev \
    libgrpc++-dev libprotobuf-dev \
    protobuf-compiler protobuf-compiler-grpc \
    build-essential g++ pkg-config
```

Verification commands:

```
pkg-config --modversion libzmq
pkg-config --modversion grpc++
pkg-config --modversion protobuf

protoc --version
which grpc_cpp_plugin
```

2.4 Build Process

Each module has an independent Makefile with no cross-dependencies, allowing the implementations to be built separately:

```
make -C shm
make -C uds
make -C zeroMQ
make -C grpc
```

2.5 Test Execution

All benchmark runs are orchestrated by `run_bench.sh`, which starts the required producer, consumer, and gRPC server processes automatically.

```
./run_bench.sh
./run_bench.sh --ipc uds --msgs 1000000 --cons 4
```

Post-test analysis can be run with:

```
python3 analyze.py --out-dir ./output --plots-dir ./output/plots \
    --ipc-list shm uds zmq grpc \
    --msg-list 100000 1000000 10000000 \
    --cons-list 1 2 4 \
    --mps-list 2000000 5000000 10000000
```

2.6 Test Case Matrix

Parameter	Values
IPC Mechanism	`shm` / `uds` / `zmq` / `grpc`
Total Messages (N)	`100,000` / `1,000,000` / `10,000,000`
Consumer Count (C)	`1` / `2` / `4`
Target Send Rate	`2M` / `5M` / `10M` msg/s

2.7 Metrics

The main metric is **P50 end-to-end latency**, expressed in microseconds in this notebook. P90 and P99 are included for context. Actual throughput is computed from the number of messages received by the consumer and the elapsed wall-clock time between producer start and consumer completion. For multi-consumer runs, consumer-level measurements are averaged within the same workload configuration before cross-workload aggregation.

3 Latency Percentile Summary by Consumer Count and Target MPS

This table reports average P50, P90, and P99 latency for each IPC mechanism, consumer count, and target MPS. Rows are grouped by `ipc + n_cons + target_mps`, so with 4 IPC mechanisms, 3 consumer-count settings, and 3 target-MPS settings, the table should contain **36 rows**.

For SHM, the reported values come from `output/shm_final.csv`, which averages multiple rounds after dropping VM-noisy outlier rows with `p50_ns > 1000`.

P50 is the primary comparison metric in this VM-based benchmark. P90 and P99 are included to show tail-latency tendency, but they should be interpreted cautiously because VM scheduling can amplify tail behavior.

```
Rows: 36 / expected 36
```

Table 1

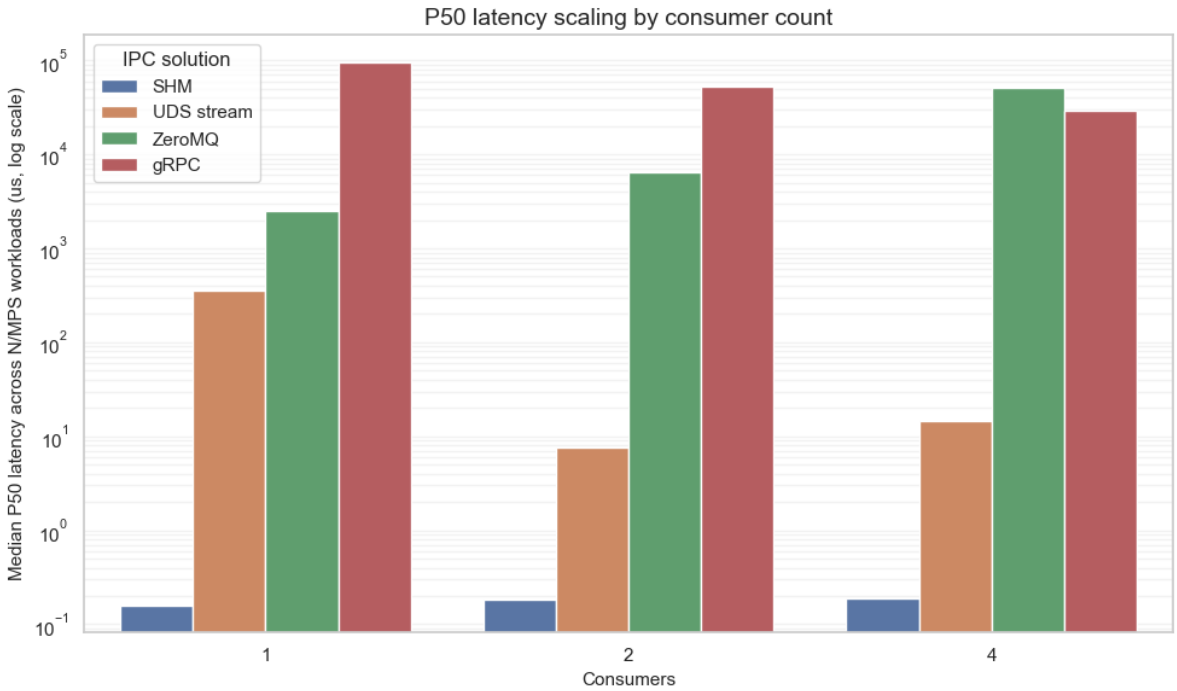
	ipc_label	n_cons	target_mps	p50_us	p90_us
0	SHM	1	2,000,000	0.16	1092.32
1	SHM	1	5,000,000	0.16	738.56
2	SHM	1	10,000,000	0.22	168.87
3	SHM	2	2,000,000	0.18	787.17
4	SHM	2	5,000,000	0.18	109.98
5	SHM	2	10,000,000	0.24	92.09
6	SHM	4	2,000,000	0.19	1025.35
7	SHM	4	5,000,000	0.19	722.84
8	SHM	4	10,000,000	0.24	315.51
9	UDS stream	1	2,000,000	350.44	653.12
10	UDS stream	1	5,000,000	372.31	648.88
11	UDS stream	1	10,000,000	348.00	644.30
12	UDS stream	2	2,000,000	7.58	156.25
13	UDS stream	2	5,000,000	7.47	101.39
14	UDS stream	2	10,000,000	7.50	128.53
15	UDS stream	4	2,000,000	13.53	164.41
16	UDS stream	4	5,000,000	17.78	494.55
17	UDS stream	4	10,000,000	14.32	228.26
18	ZeroMQ	1	2,000,000	494.23	1582.34
19	ZeroMQ	1	5,000,000	9507.11	11740.21
20	ZeroMQ	1	10,000,000	4132.68	6930.39
21	ZeroMQ	2	2,000,000	4877.93	8048.88
22	ZeroMQ	2	5,000,000	7717.01	10686.49

	ipc_label	n_cons	target_mps	p50_us	p90_us
23	ZeroMQ	2	10,000,000	10921.70	18721.20
24	ZeroMQ	4	2,000,000	45948.30	63775.40
25	ZeroMQ	4	5,000,000	46638.14	66563.71
26	ZeroMQ	4	10,000,000	56847.43	80917.92
27	gRPC	1	2,000,000	94008.84	120234.72
28	gRPC	1	5,000,000	94078.18	125694.64
29	gRPC	1	10,000,000	96571.69	131043.63
30	gRPC	2	2,000,000	51303.36	88491.80
31	gRPC	2	5,000,000	52042.80	87952.37
32	gRPC	2	10,000,000	48138.77	86108.44
33	gRPC	4	2,000,000	30236.18	80545.16
34	gRPC	4	5,000,000	29908.52	79475.64
35	gRPC	4	10,000,000	33220.34	81886.23

4 P50 Latency Scaling by Consumer Count

This figure focuses on the main benchmark question: how typical latency changes as the number of consumers increases. For each `ipc + n_msgs + target_mps + n_cons` configuration, P50 values from multiple consumers are averaged first. Then, for each `ipc + n_cons`, the median across different message counts and target MPS values is used. This avoids overweighting C=2 or C=4 runs simply because they contain more consumer rows.

A lower bar indicates better typical latency. Since the y-axis is logarithmic, differences between mechanisms can span multiple orders of magnitude.



ipc_label n_cons	SHM	UDS stream	ZeroMQ	gRPC
1	0.158882	352.800445	2483.337607	95464.077769
2	0.181602	7.516520	6468.448255	52016.682226
4	0.188493	14.473689	51124.220321	29051.460896

4.1 Interpretation of the UDS and gRPC Consumer-Count Dip

UDS stream can show lower P50 latency at C=2 than at C=1 or C=4. This does not mean that adding consumers makes the IPC mechanism intrinsically faster. With C=1, the producer only writes to one FD, so at high target rates it can push data into the socket buffer faster than the consumer drains it; the measured latency then includes more queueing time. With C=4, each logical message requires four sequential fan-out sends, which increases producer-side work, scheduling pressure, and socket overhead. C=2 sits between these effects: the extra fan-out and kernel backpressure can keep the socket queue shallower, while the fan-out cost is still lower than C=4.

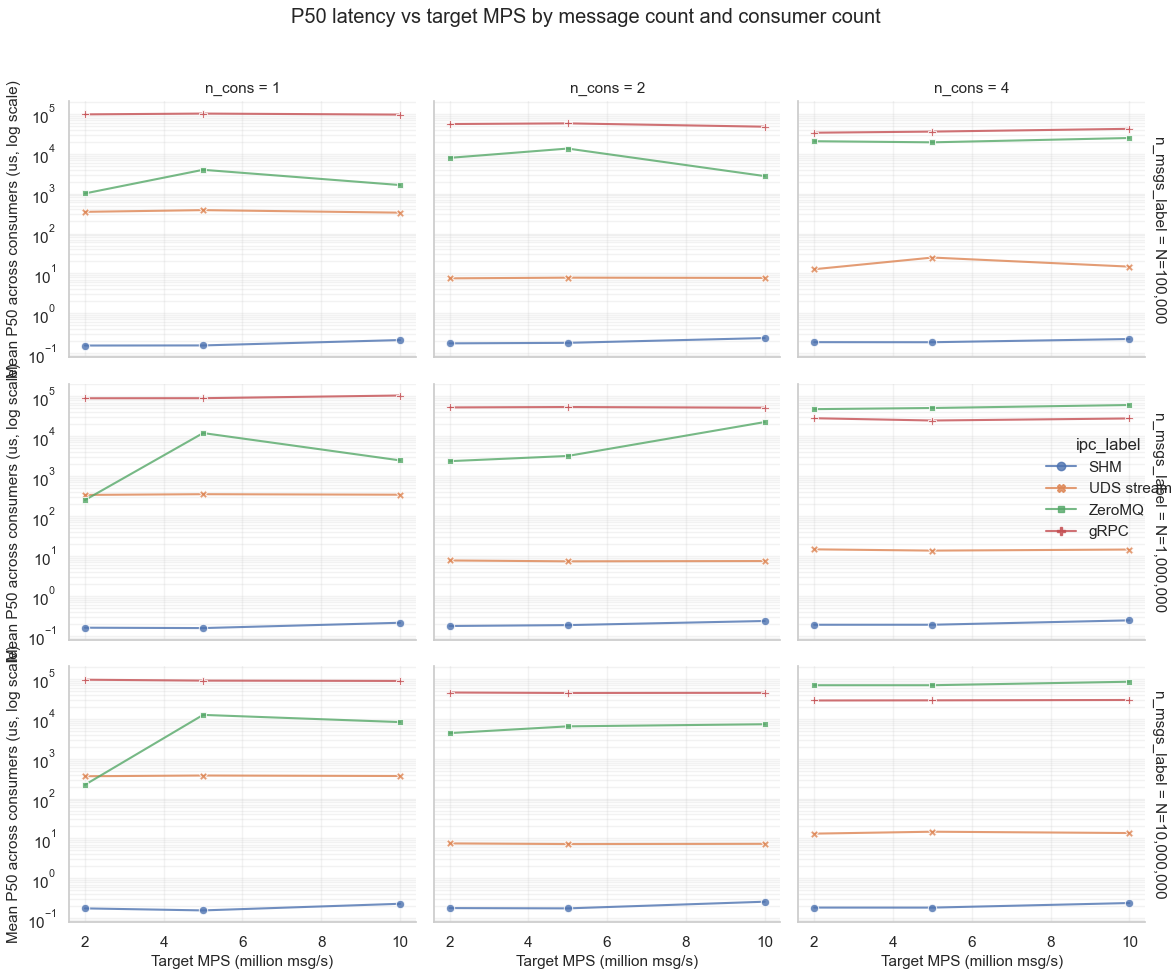
gRPC can show a similar pattern for the same queueing/backpressure reason, with extra effects from the RPC stack. A single stream may allow the server to enqueue work quickly and build up queueing delay, while multiple streams introduce flow control, completion-queue scheduling, serialization/deserialization work, and per-consumer fan-out cost. At moderate consumer count

this can reduce queue buildup and lower P50, but as consumer count keeps increasing the additional RPC and scheduling overhead becomes dominant again.

5 P50 Latency vs Target MPS at Fixed Consumer Counts

This plot keeps consumer count fixed in each column and message count fixed in each row, then shows how P50 latency changes as the target send rate increases. Multiple consumers in the same configuration are averaged before plotting, so a C=4 configuration contributes one point per IPC mechanism and target MPS.

This view helps separate fan-out scaling from send-rate pressure. If a mechanism is close to saturation, P50 latency may increase as the target MPS rises. If the producer cannot reach the requested rate, the actual-throughput section should be used together with this plot.



6 Throughput by Target MPS and Consumer Count

This section reports actual measured throughput. For each `ipc + n_msgs + target_mps + n_cons` configuration, throughput values from multiple consumers are averaged. The table includes the mean throughput in messages per second and the percentage of the target rate achieved.

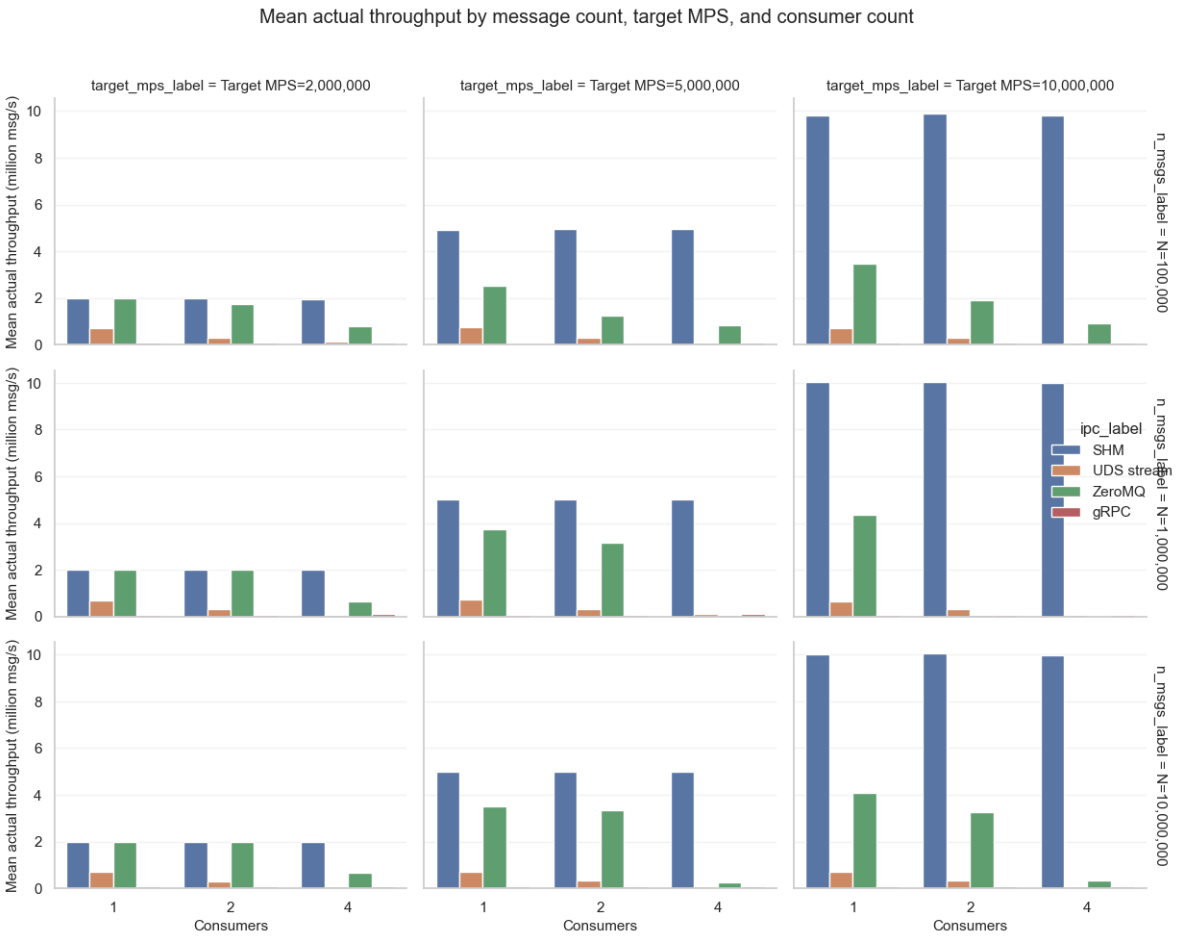
The bar charts show actual throughput and target-achievement percentage across message counts, target MPS values, and consumer counts. UDS stream and gRPC may appear below SHM on a linear throughput scale because they pay kernel socket, copy, framing, or RPC-stack overhead that SHM avoids.

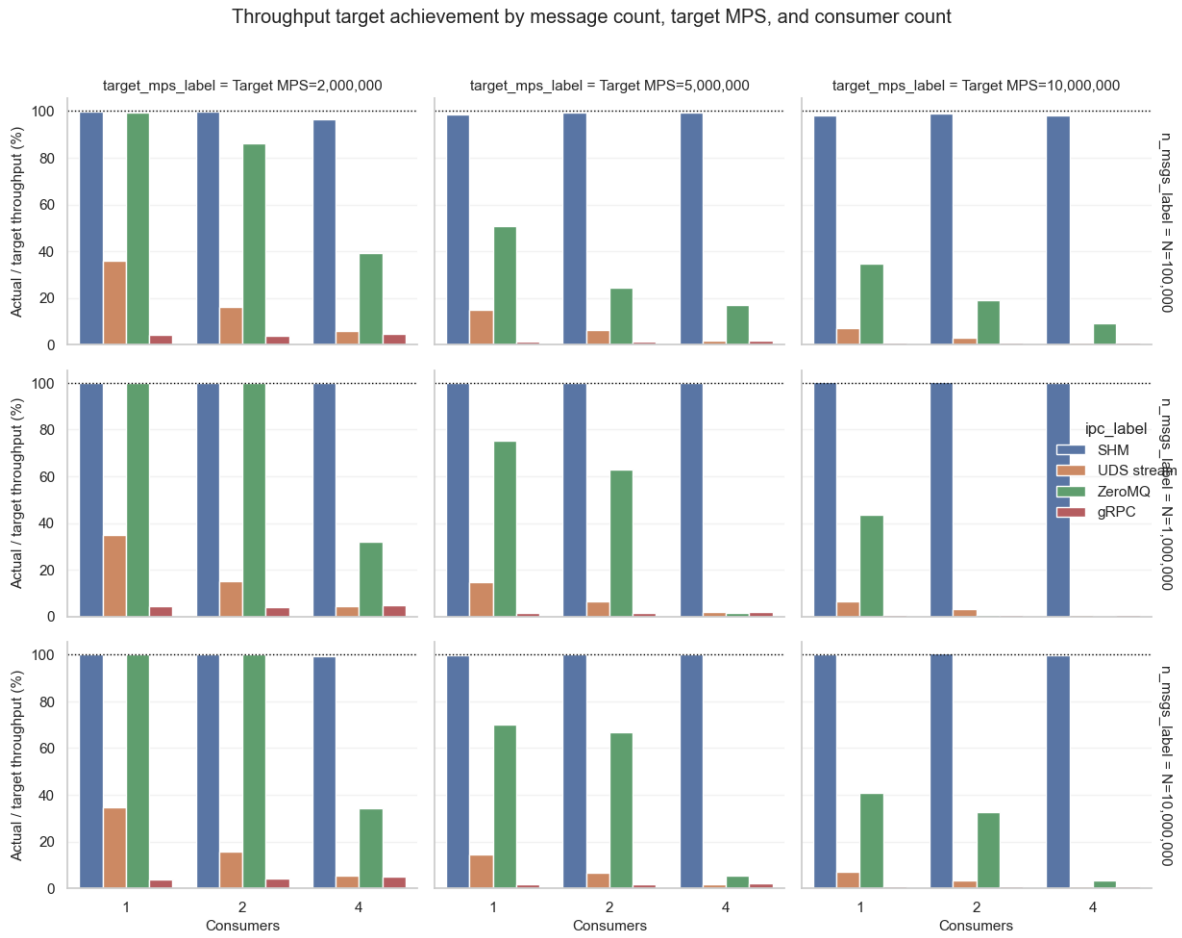
Table 2

	ipc_label	n_msgs	target_mps	n_cons	throughput_mps_mean	target_achievement_pct
0	SHM	100,000	2,000,000	1	1,995,896	99.79%
1	SHM	100,000	2,000,000	2	1,994,705	99.74%
2	SHM	100,000	2,000,000	4	1,932,076	96.60%
3	SHM	100,000	5,000,000	1	4,923,023	98.46%
4	SHM	100,000	5,000,000	2	4,965,179	99.30%
5	SHM	100,000	5,000,000	4	4,962,386	99.25%
6	SHM	100,000	10,000,000	1	9,824,811	98.25%
7	SHM	100,000	10,000,000	2	9,902,945	99.03%
8	SHM	100,000	10,000,000	4	9,793,731	97.94%
9	SHM	1,000,000	2,000,000	1	2,001,039	100.05%
10	SHM	1,000,000	2,000,000	2	2,000,886	100.04%
11	SHM	1,000,000	2,000,000	4	1,999,525	99.98%
12	SHM	1,000,000	5,000,000	1	4,999,899	100.00%
13	SHM	1,000,000	5,000,000	2	5,006,446	100.13%
14	SHM	1,000,000	5,000,000	4	4,996,132	99.92%
15	SHM	1,000,000	10,000,000	1	10,025,709	100.26%
16	SHM	1,000,000	10,000,000	2	10,025,449	100.25%
17	SHM	1,000,000	10,000,000	4	9,972,596	99.73%
18	SHM	10,000,000	2,000,000	1	2,001,547	100.08%
19	SHM	10,000,000	2,000,000	2	2,000,047	100.00%
20	SHM	10,000,000	2,000,000	4	1,982,949	99.15%
21	SHM	10,000,000	5,000,000	1	4,979,627	99.59%
22	SHM	10,000,000	5,000,000	2	5,009,692	100.19%
23	SHM	10,000,000	5,000,000	4	5,005,936	100.12%
24	SHM	10,000,000	10,000,000	1	10,030,168	100.30%
25	SHM	10,000,000	10,000,000	2	10,033,743	100.34%
26	SHM	10,000,000	10,000,000	4	9,989,795	99.90%
27	UDS stream	100,000	2,000,000	1	719,008	35.95%

	ipc_label	n_msgs	target_mps	n_cons	throughput_mps_mean	target_achievement_pct
28	UDS stream	100,000	2,000,000	2	320,539	16.03%
29	UDS stream	100,000	2,000,000	4	117,215	5.86%
30	UDS stream	100,000	5,000,000	1	747,994	14.96%
31	UDS stream	100,000	5,000,000	2	310,006	6.20%
32	UDS stream	100,000	5,000,000	4	83,624	1.67%
33	UDS stream	100,000	10,000,000	1	704,693	7.05%
34	UDS stream	100,000	10,000,000	2	305,204	3.05%
35	UDS stream	100,000	10,000,000	4	106,488	1.06%
36	UDS stream	1,000,000	2,000,000	1	694,220	34.71%
37	UDS stream	1,000,000	2,000,000	2	300,389	15.02%
38	UDS stream	1,000,000	2,000,000	4	91,390	4.57%
39	UDS stream	1,000,000	5,000,000	1	728,618	14.57%
40	UDS stream	1,000,000	5,000,000	2	315,359	6.31%
41	UDS stream	1,000,000	5,000,000	4	107,613	2.15%
42	UDS stream	1,000,000	10,000,000	1	666,254	6.66%
43	UDS stream	1,000,000	10,000,000	2	317,845	3.18%
44	UDS stream	1,000,000	10,000,000	4	92,028	0.92%
45	UDS stream	10,000,000	2,000,000	1	692,190	34.61%
46	UDS stream	10,000,000	2,000,000	2	311,407	15.57%
47	UDS stream	10,000,000	2,000,000	4	109,686	5.48%
48	UDS stream	10,000,000	5,000,000	1	725,439	14.51%
49	UDS stream	10,000,000	5,000,000	2	325,995	6.52%
50	UDS stream	10,000,000	5,000,000	4	96,767	1.94%
51	UDS stream	10,000,000	10,000,000	1	721,042	7.21%
52	UDS stream	10,000,000	10,000,000	2	321,073	3.21%
53	UDS stream	10,000,000	10,000,000	4	108,846	1.09%
54	ZeroMQ	100,000	2,000,000	1	1,986,980	99.35%
55	ZeroMQ	100,000	2,000,000	2	1,721,108	86.06%
56	ZeroMQ	100,000	2,000,000	4	787,563	39.38%
57	ZeroMQ	100,000	5,000,000	1	2,532,528	50.65%
58	ZeroMQ	100,000	5,000,000	2	1,228,659	24.57%
59	ZeroMQ	100,000	5,000,000	4	852,333	17.05%
60	ZeroMQ	100,000	10,000,000	1	3,483,398	34.83%
61	ZeroMQ	100,000	10,000,000	2	1,888,455	18.88%
62	ZeroMQ	100,000	10,000,000	4	925,027	9.25%
63	ZeroMQ	1,000,000	2,000,000	1	1,999,385	99.97%
64	ZeroMQ	1,000,000	2,000,000	2	1,998,349	99.92%
65	ZeroMQ	1,000,000	2,000,000	4	639,423	31.97%
66	ZeroMQ	1,000,000	5,000,000	1	3,751,930	75.04%
67	ZeroMQ	1,000,000	5,000,000	2	3,153,309	63.07%
68	ZeroMQ	1,000,000	5,000,000	4	68,665	1.37%

	ipc_label	n_msgs	target_mps	n_cons	throughput_mps_mean	target_achievement_pct
69	ZeroMQ	1,000,000	10,000,000	1	4,355,972	43.56%
70	ZeroMQ	1,000,000	10,000,000	2	89,437	0.89%
71	ZeroMQ	1,000,000	10,000,000	4	27,687	0.28%
72	ZeroMQ	10,000,000	2,000,000	1	2,001,337	100.07%
73	ZeroMQ	10,000,000	2,000,000	2	2,000,952	100.05%
74	ZeroMQ	10,000,000	2,000,000	4	685,194	34.26%
75	ZeroMQ	10,000,000	5,000,000	1	3,495,495	69.91%
76	ZeroMQ	10,000,000	5,000,000	2	3,335,732	66.71%
77	ZeroMQ	10,000,000	5,000,000	4	272,006	5.44%
78	ZeroMQ	10,000,000	10,000,000	1	4,072,844	40.73%
79	ZeroMQ	10,000,000	10,000,000	2	3,247,272	32.47%
80	ZeroMQ	10,000,000	10,000,000	4	358,062	3.58%
81	gRPC	100,000	2,000,000	1	83,050	4.15%
82	gRPC	100,000	2,000,000	2	76,025	3.80%
83	gRPC	100,000	2,000,000	4	89,318	4.47%
84	gRPC	100,000	5,000,000	1	69,874	1.40%
85	gRPC	100,000	5,000,000	2	70,917	1.42%
86	gRPC	100,000	5,000,000	4	86,327	1.73%
87	gRPC	100,000	10,000,000	1	78,630	0.79%
88	gRPC	100,000	10,000,000	2	82,519	0.83%
89	gRPC	100,000	10,000,000	4	88,601	0.89%
90	gRPC	1,000,000	2,000,000	1	90,272	4.51%
91	gRPC	1,000,000	2,000,000	2	76,410	3.82%
92	gRPC	1,000,000	2,000,000	4	94,036	4.70%
93	gRPC	1,000,000	5,000,000	1	86,027	1.72%
94	gRPC	1,000,000	5,000,000	2	76,454	1.53%
95	gRPC	1,000,000	5,000,000	4	94,600	1.89%
96	gRPC	1,000,000	10,000,000	1	74,361	0.74%
97	gRPC	1,000,000	10,000,000	2	79,451	0.79%
98	gRPC	1,000,000	10,000,000	4	90,620	0.91%
99	gRPC	10,000,000	2,000,000	1	80,255	4.01%
100	gRPC	10,000,000	2,000,000	2	82,600	4.13%
101	gRPC	10,000,000	2,000,000	4	97,994	4.90%
102	gRPC	10,000,000	5,000,000	1	85,249	1.70%
103	gRPC	10,000,000	5,000,000	2	85,244	1.70%
104	gRPC	10,000,000	5,000,000	4	97,942	1.96%
105	gRPC	10,000,000	10,000,000	1	83,800	0.84%
106	gRPC	10,000,000	10,000,000	2	82,376	0.82%
107	gRPC	10,000,000	10,000,000	4	99,055	0.99%

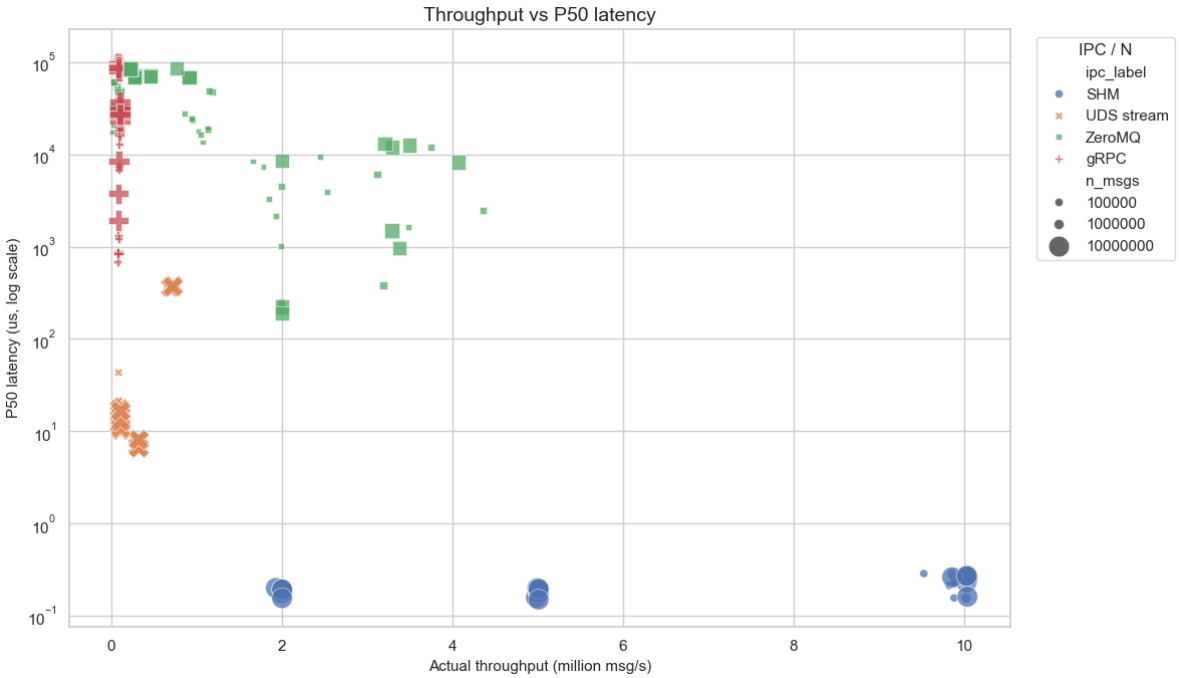

Mean actual throughput by message count, target MPS, and consumer count
target_mps_label = Target MPS=2,000,000
target_mps_label = Target MPS=5,000,000
target_mps_label = Target MPS=10,000,000
n_msgs_label = N=100,000
n_msgs_label = N=1,000,000
n_msgs_label = N=10,000,000
Mean actual throughput (million msg/s)
0
2
4
6
8
10
Consumers
1
2
4
ipc_label
SHM
UDS stream
ZeroMQ
gRPC



7 Throughput vs P50 Latency

This scatter plot relates capacity and typical latency. The x-axis is actual measured throughput, and the y-axis is P50 latency. The ideal region is toward the lower-right corner: high throughput with low typical latency.

Because the x-axis uses actual throughput rather than target MPS, mechanisms that cannot reach the requested send rate will appear further left. This is important for interpreting UDS stream, ZeroMQ, and gRPC results: a low latency number is less meaningful if the achieved throughput is far below the target workload.



8 Loss by Target MPS

This table reports message-delivery loss aggregated by IPC mechanism and target send rate across all tested message counts and consumer counts. Loss is included as a reference metric rather than the primary ranking metric. This benchmark models local market-data fan-out, closer to latest-value dissemination than durable message persistence. In such pipelines, occasional dropped updates may be acceptable if sequence gaps are detectable and consumers can resynchronize from a newer snapshot or stream position. This does not apply to order, execution, or risk-control paths, where reliable delivery is required.

Table 3

	ipc_label	target_mps	expected_messages	received_messages	lost_messages	loss_rate_pct
0	SHM	2,000,000	77,700,000	77,341,156	358,844	0.4618%
1	SHM	5,000,000	77,700,000	77,619,387	80,613	0.1037%
2	SHM	10,000,000	77,700,000	77,488,209	211,791	0.2726%
3	UDS stream	2,000,000	77,700,000	77,700,000	0	0.0000%
4	UDS stream	5,000,000	77,700,000	77,700,000	0	0.0000%
5	UDS stream	10,000,000	77,700,000	77,700,000	0	0.0000%
6	ZeroMQ	2,000,000	77,700,000	54,741,827	22,958,173	29.5472%
7	ZeroMQ	5,000,000	77,700,000	41,430,081	36,269,919	46.6794%

	ipc_label	target_mps	expected_messages	received_messages	lost_messages	loss_rate_pct
8	ZeroMQ	10,000,000	77,700,000	38,216,788	39,483,212	50.8149%
9	gRPC	2,000,000	77,700,000	77,700,000	0	0.0000%
10	gRPC	5,000,000	77,700,000	77,700,000	0	0.0000%
11	gRPC	10,000,000	77,700,000	77,700,000	0	0.0000%

9 Result Summary and Conclusion

9.1 Executive Summary

Fastest Overall: SHM

The custom lock-free shared-memory implementation remains the primary low-latency reference point. It avoids kernel socket copies and per-consumer send syscalls on the producer hot path, so it is expected to dominate the in-host fan-out workload when the custom shared-memory lifecycle is acceptable.

Direct Kernel-Socket Baseline: UDS stream

UDS stream provides a useful baseline between custom SHM and higher-level messaging stacks. It keeps the same 64-byte `Message` layout and uses one local stream connection per consumer, but the producer still performs one `send_all()` per consumer and each consumer reconstructs fixed-size messages with `recv_exact()`.

General Messaging Baseline: ZeroMQ

ZeroMQ remains the strongest general messaging-library baseline when a ready-made PUB/SUB abstraction is preferred. Its behavior under high target rates should be interpreted together with achieved throughput and loss.

Highest-Abstraction Baseline: gRPC

gRPC is useful for typed service communication and server-streaming APIs, but its serialization, RPC scheduling, and stream-management costs make it the least directly comparable to the 64-byte local fan-out fast path.

9.2 Key Findings

- **SHM is the clear winner for in-host HFT-style fan-out.** It is the only implementation in this set that avoids per-message kernel socket send/receive overhead.
- **UDS stream is the right low-level socket baseline.** It gives every consumer a dedicated connection and preserves fixed-size message framing with explicit `send_all()` / `recv_exact()` handling.

- **Consumer count is a first-class scaling dimension.** The benchmark is not only a single-consumer latency test; it measures how each mechanism behaves as fan-out increases from C=1 to C=2 and C=4.
- **ZeroMQ is the strongest standard messaging baseline, but not a replacement for SHM latency.** Its library-level PUB/SUB abstraction is convenient, while overloaded cases must be checked for loss and throughput shortfall.
- **gRPC is convenient but not latency-competitive here.** It is valuable for typed service communication, but its abstraction cost is too high for this 64-byte local market-data fan-out workload.
- **VM results should be interpreted conservatively.** P50 is the primary metric in this report. P90/P99 are useful for trend inspection, but VM scheduling and host interference can distort tail-latency measurements.

9.3 Recommendations

- **Use SHM** when the objective is minimum latency and maximum local throughput, and when the application can tolerate custom shared-memory lifecycle and synchronization code.
- **Use UDS stream** when you want a simple kernel-supported local IPC baseline with explicit one-connection-per-consumer fan-out and reliable byte-stream delivery.
- **Use ZeroMQ** when a standard messaging library is preferred and throughput matters more than sub-microsecond latency, while carefully handling high-load loss behavior.
- **Use gRPC** for service APIs and typed streaming interfaces, not for ultra-low-latency market-data distribution.
- **Repeat on bare metal** for final HFT claims. A bare-metal Linux run with CPU pinning, isolated cores, and hardware performance counters would provide more reliable P99 and micro-architectural measurements.